

Singly Linked Lists: Breaking Free of Contiguous Memory

Programming and Data Structures — Week 4, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain why a linked list's nodes do not need to sit next to each other in memory, unlike every structure covered so far this course; implement a `Node` class and a singly linked `LinkedList` entirely from scratch; state the time complexity of insertion at the head, insertion at the tail, and search, and justify each cost from the structure's mechanics rather than by memorization; and trace the exact pointer updates involved in inserting and deleting a node, step by step.

Outline

Today covers a treasure-hunt analogy that reframes the apartment-building picture from Week 2; the precise meaning of a node and of “linked”; why traversal is the only way to reach an element in this structure; insertion at the head, traced fully; insertion at the tail and the extra care it requires; deletion, also traced fully; a cost summary set directly against Week 2's array cost table for comparison; an in-class quiz; and a summary.

The treasure-hunt analogy

Week 2 introduced arrays using an apartment-building analogy, where any flat's address could be computed directly from its number. A linked list is the opposite: a treasure hunt, where each clue tells you only where to find the *next* clue, never where clue 47 specifically sits. Reaching clue 47 genuinely requires visiting clues 1 through 46 first, in order — there is no shortcut, no formula, no direct computation. This is exactly the trade-off Week 2 Lecture 1 flagged as coming: giving up $O(1)$ index access in exchange for something else, which this lecture now makes concrete.

What a node is

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None    # a reference to the NEXT node, or None if last
```

A node is a small object holding exactly two things: its own data (`value`) and a reference to the next node in the chain (`next`). Unlike an array's elements, nodes are not required to sit next to each other in memory at all — each node is created as a separate, independent object, and the `next` reference is what stitches them together into a single logical sequence, regardless of where each one physically lives. Setting `self.next = None` marks the end of the chain: there is no node after the last one.

Building the chain by hand

```
first = Node(10)
first.next = Node(20)
first.next.next = Node(30)

print(first.value)           # 10
print(first.next.value)     # 20
print(first.next.next.value) # 30
print(first.next.next.next) # None - end of the chain
```

Building a three-node chain by hand makes the mechanism explicit: `first` is a `Node` holding 10, whose `next` points to a second node holding 20, whose `next` in turn points to a third node holding 30, whose `next` is `None`. Each `.next` in the code is a genuine hop across memory, not an arithmetic computation — this is the structural opposite of Week 2's `address(i) = base + i * element_size`, which needed no hopping at all.

The `LinkedList` wrapper class

```
class LinkedList:
    def __init__(self):
        self.head = None    # reference to the first node, or None if empty
        self._size = 0
```

```

def __len__(self):
    return self._size

def is_empty(self):
    return self.head is None

```

A `LinkedList` wrapper class holds a single reference, `head`, to the first node in the chain — every other node is reached only by following `.next` references starting from `head`. Tracking `_size` explicitly as a separate counter avoids ever needing to traverse the whole list just to answer `len()`, mirroring the same design instinct Week 3's circular buffer used for tracking count explicitly rather than re-deriving it.

Traversal: the only way to reach an element

```

def to_list(self):
    result = []
    current = self.head
    while current is not None:
        result.append(current.value)
        current = current.next
    return result

```

Traversal starts at `head` and follows `.next` references one at a time until reaching `None`, which marks the end. This is the *only* way to reach any node in a singly linked list — there is no equivalent of an array's direct-address `arr[i]`. Reaching the k -th node costs $O(k)$, since k hops must be followed in sequence, and traversing the entire list costs $O(n)$. This is the direct structural cost of giving up contiguous memory: what was $O(1)$ for an array (`arr[i]`) becomes $O(n)$ in the worst case here.

Insertion at the head: the cheap case

```

def push_front(self, value):
    new_node = Node(value)
    new_node.next = self.head    # new node points to the OLD head
    self.head = new_node        # head now points to the new node
    self._size += 1

```

Inserting a new node at the front of the list requires exactly two reference assignments: the new node's `next` is set to point at whatever the current head is, and then `head` itself is updated to point at

the new node. Critically, nothing about any existing node changes — no values move, no memory is copied, regardless of whether the list holds 3 nodes or 3 million. The cost is genuinely $O(1)$, and unlike a dynamic array's amortized append, this is not an average over time — every single `push_front` call costs exactly the same small, constant amount.

Tracing head-insertion, step by step

Before: `head` → [10] → [20] → [30] → None

`push_front(5)`:

Step 1: `new_node.next = head` → `new_node(5)` → [10] → [20] → [30] → None

Step 2: `head = new_node` → `head` → [5] → [10] → [20] → [30] → None

Tracing `push_front(5)` on a list already holding 10, 20, 30: step 1 creates the new node holding 5 and points its `next` at the current head (the node holding 10) — at this instant, two structures technically coexist, the old list and the new node pointing into it. Step 2 then updates `head` itself to point at the new node, which is the single assignment that actually makes the new node “part of” the list from the outside. Notice the *order* matters: setting `head` before setting `new_node.next` would lose the reference to the rest of the list entirely.

Insertion at the tail: why it needs more care

```
def push_back(self, value):
    new_node = Node(value)
    if self.head is None:
        self.head = new_node
        self._size += 1
        return
    current = self.head
    while current.next is not None:      # walk to the LAST node
        current = current.next
    current.next = new_node
    self._size += 1
```

Inserting at the tail is structurally different from inserting at the head, because a plain singly linked list keeps a direct reference only to the first node, `head` — there is no equivalent `tail` reference by default. Reaching the last node to attach a new one requires traversing the entire list first, an $O(n)$ operation. This is a genuine, unavoidable cost of this particular design, not a minor technicality — and it motivates the “maintain an explicit `tail` reference” optimization covered later this week, once doubly linked lists are introduced.

Tracing tail-insertion, step by step

Before: head → [10] → [20] → [30] → None

push_back(40):

```
current = head (node 10)
current.next is [20], not None → current = [20]
current.next is [30], not None → current = [30]
current.next is None → stop here, current is the last node
current.next = new_node(40)
```

After: head → [10] → [20] → [30] → [40] → None

Tracing push_back(40): the traversal begins at head and walks forward one node at a time, checking .next at each step, until reaching a node whose .next is None — the current last node, holding 30. Only then is the new node attached, by setting that last node's .next to point at it. For a list of length n , this traversal visits all n existing nodes before the insertion itself can happen, which is exactly where the $O(n)$ cost comes from.

Deletion: removing a value

```
def remove(self, value):
    if self.head is None:
        return False
    if self.head.value == value:
        self.head = self.head.next    # skip over the old head entirely
        self._size -= 1
        return True
    current = self.head
    while current.next is not None:
        if current.next.value == value:
            current.next = current.next.next    # skip over the target node
            self._size -= 1
            return True
        current = current.next
    return False
```

Deleting the head node requires only reassigning head to skip over it — $O(1)$, and the old head node becomes unreachable and is garbage collected. Deleting any other node is more involved: the code must find the node *immediately before* the target, since that is the only node whose .next reference

can be redirected to skip over the target. This search costs $O(n)$ in the worst case; the actual unlinking, once the right predecessor node is found, is a single $O(1)$ reference reassignment.

Tracing deletion, step by step

Before: `head → [10] → [20] → [30] → None`

`remove(20):`

`head.value (10) != 20 → current = head (node 10)`

`current.next.value (20) == 20 → MATCH`

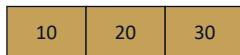
`current.next = current.next.next` (skip node 20, point straight to node 30)

After: `head → [10] → [30] → None` (node 20 is now unreachable)

Tracing `remove(20)`: the head itself, holding 10, does not match, so the search proceeds to check `current.next` for a match at each step. It finds that `current.next.value (20)` matches, and unlinks it with a single reassignment: `current.next = current.next.next`, which redirects the node holding 10 to point directly at the node holding 30, skipping over 20 entirely. The node holding 20 still technically exists in memory for a moment, but nothing in the list references it anymore, so it becomes unreachable and is eventually reclaimed.

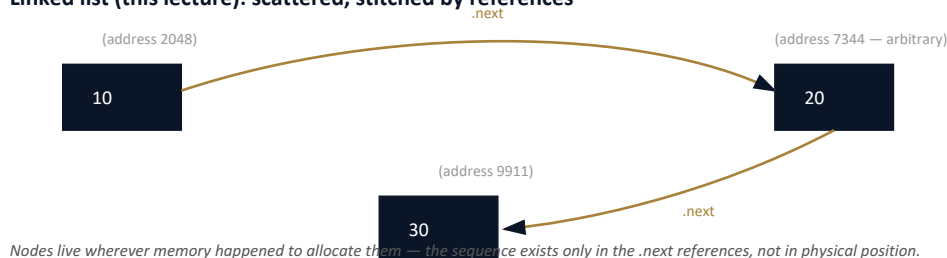
The picture: nodes scattered, links stitching them together

Array (Week 2): contiguous, one block



addresses 1000, 1004, 1008 — back to back, `address(i)` computed directly

Linked list (this lecture): scattered, stitched by references



Drawing the nodes scattered rather than lined up, connected only by arrows, is deliberately different from every array diagram so far this course — nodes genuinely need not be near each other in memory, and the “list” structure exists entirely in the chain of `.next` references, not in physical proximity.

Cost summary, contrasted with Week 2's array table

Placing this lecture's results directly against Week 2's array cost table makes the trade-off unmistakable: a linked list is *worse* at index access ($O(n)$ instead of array's $O(1)$) and *worse* at end-insertion ($O(n)$ instead of amortized $O(1)$), but *better* at front-insertion (genuine $O(1)$ instead of array's $O(n)$ shifting cost). Neither structure dominates the other — the correct choice depends entirely on which operations a given program performs most, exactly the same lesson Week 2 Lecture 4 drew when comparing `list` against `deque`.

Why Python's own `list` is not a linked list

It is worth explicitly correcting a common point of confusion: despite its name, Python's built-in `list` type is the dynamic array analyzed in Week 2, not a linked list in the sense of this lecture. `collections.deque` sits closer in spirit to a linked structure, implemented internally as a doubly linked list of fixed-size blocks. The `LinkedList` class built in this lecture is a genuinely separate, from-scratch structure, worth building by hand precisely because Python's standard library does not expose a plain singly linked list directly — understanding it requires implementing it.

Exercise

As an exercise, implement `LinkedList.get(i)`, returning the value stored at position i by traversing from `head`. Separately, implement `LinkedList.reverse()`, which reverses the list in place — every node stays the same object, only the direction of the `.next` references changes — with no new nodes created. Before running your implementation, trace it by hand on the three-node list `[10] → [20] → [30]`, tracking exactly how each node's `.next` reference changes at each step.

Quiz — predict the cost

For a singly linked list holding 1,000 nodes, rank these three operations from cheapest to most expensive, before checking the answer: inserting at the front; inserting at the back; and removing the value stored in the very last node.

Quiz — answer

`push_front(x)` is cheapest, at genuine $O(1)$, since it touches only the head reference. `push_back(x)` and removing the value stored in the last node are tied, both $O(n)$: `push_back` must traverse to find the current last node before attaching a new one, and removing the last node's value must traverse to find the *second-to-last* node (the one whose `.next` needs updating) before it can unlink the target — both require walking nearly the entire list first, for the same underlying reason: this list keeps a reference only to head, not to the tail.

Summary

A node holds its own data plus a reference to the next node, and unlike an array, nodes need not sit contiguously in memory — the “list” exists entirely in the chain of references. Traversal, following `.next` from `head`, is the only way to reach a node; there is no $O(1)$ index-access equivalent to an array’s address formula. Insertion and deletion at the head are genuinely $O(1)$, while the same operations at the tail cost $O(n)$ without an explicit `tail` reference, since reaching the last node requires a full traversal. This trade-off — $O(1)$ front operations in exchange for $O(n)$ access and tail operations — is structural and deliberate, not a flaw to be fixed; choosing between an array and a linked list means choosing which of these costs a given program can better afford. Next lecture adds a `prev` reference to each node, building the doubly linked list, and examines exactly what that second reference buys.